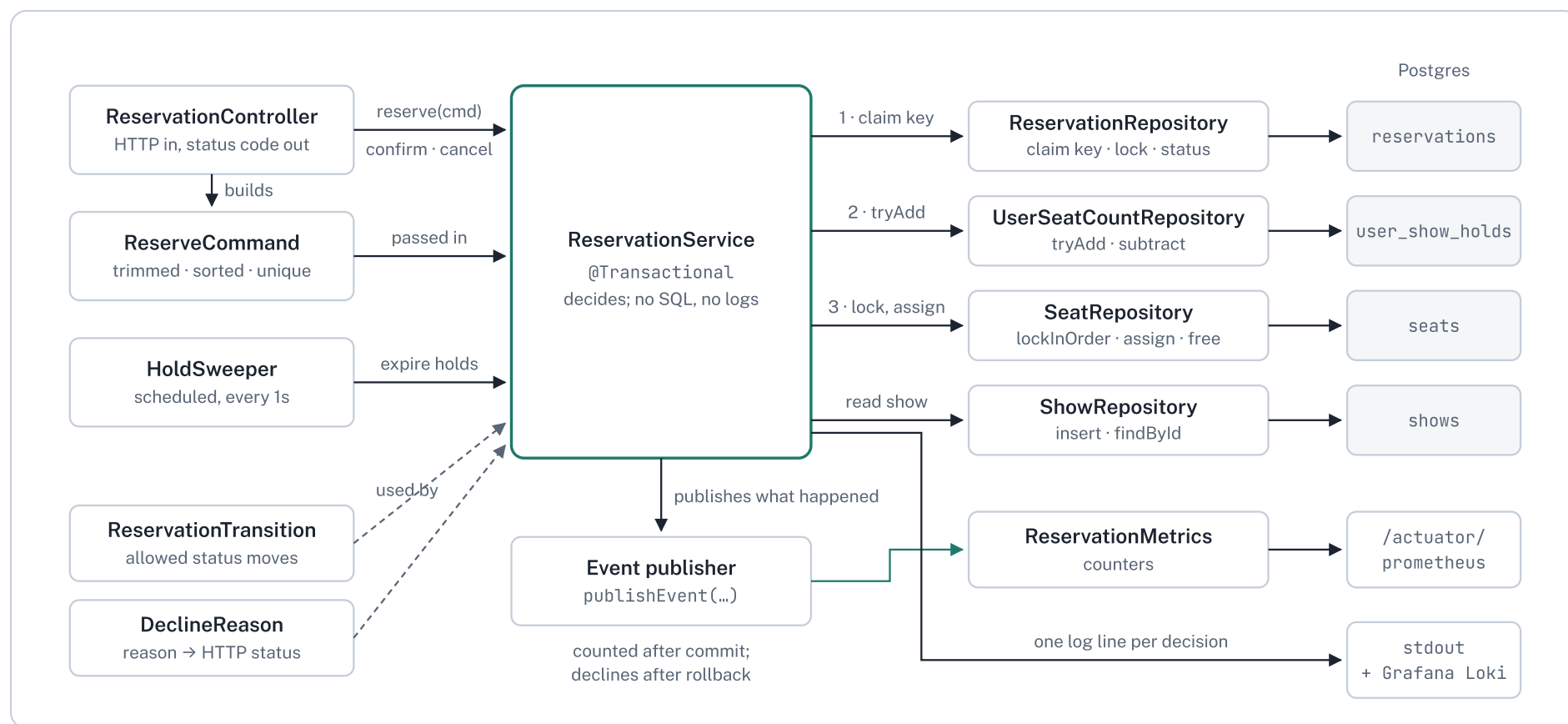


SEAT_SELECTION · AFTER THE REFACTOR

Reservation Flow Map

How a reservation request moves through the code: who calls whom, the order in which reserve() takes its locks, which outcomes commit or roll back, and the states a reservation can be in.

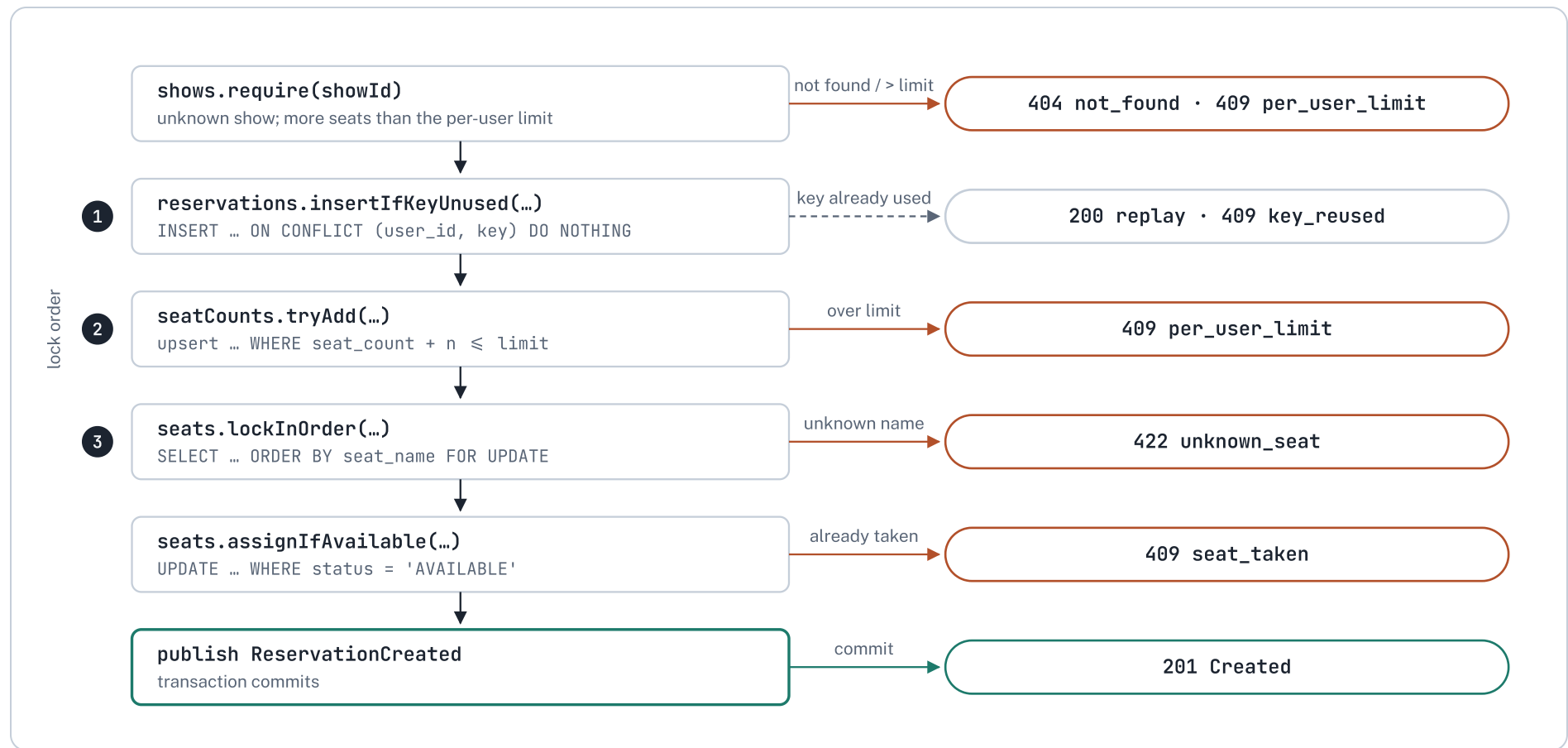
Who talks to whom



The service only decides. All SQL lives in the four repositories, and the service calls them in a fixed order (1, 2, 3): reservation row, the user's seat count, then seats in name order. That order is the lock

order on every path, so concurrent requests cannot deadlock. The service writes one structured log line per decision itself. Metrics hang off the event publisher, so the counters only ever count what the database actually committed.

reserve(), step by step



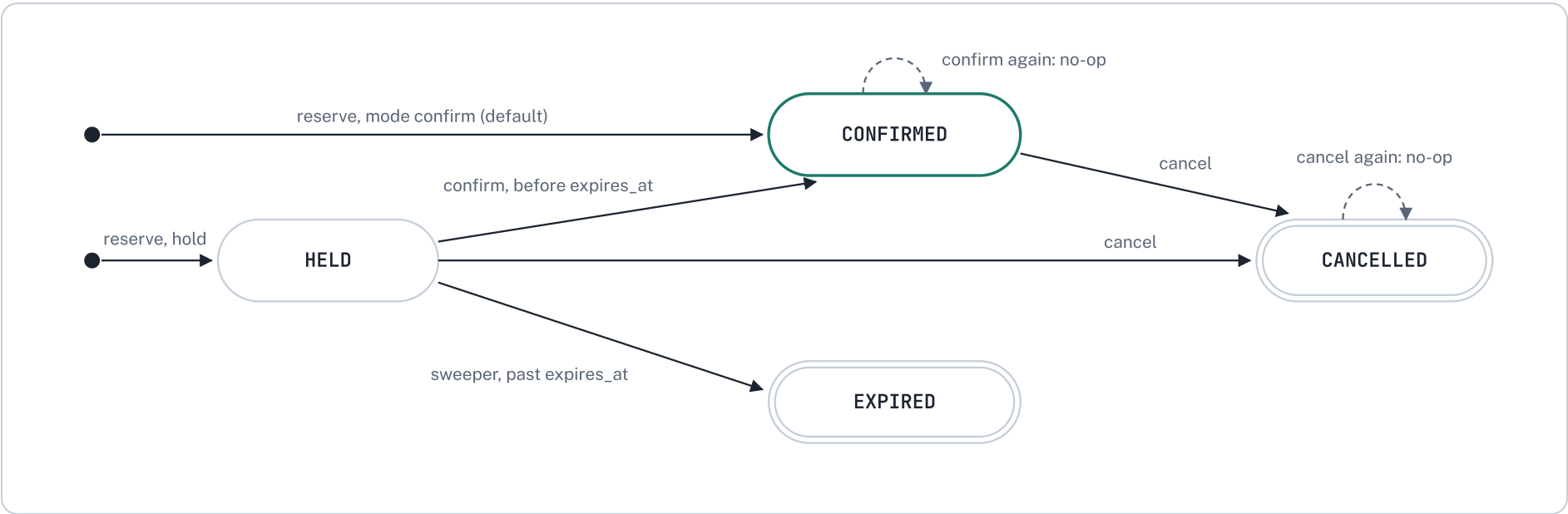
— throws: everything rolls back, counted after rollback — commits, counted after commit

--- same request again: 200 with the original; different request: 409

All six steps run in one transaction, in this order. Steps 1–3 take row locks, always in this order; a waiter on a hot seat re-reads the row after the winner commits and leaves at step 5. Because every exit rolls back, a declined request leaves nothing behind: no reservation row, no change to the user's

seat count, and its idempotency key is free for a retry.

Reservation lifecycle



The allowed moves, as written in ReservationTransition. Double borders mark final states. Repeating a move that already happened returns the reservation unchanged, so a client can safely retry confirm or cancel. Any other move is a 409 not_active, for example confirming a hold the sweeper already expired. Every release frees seats only where reservation_id matches, so it can never free a seat someone else now holds. The sweeper runs every second; if it ever stalls, the reservations_holds_overdue gauge (holds still held 10s past their deadline) rises above 0, which is the alert to page on.

Where each piece lives

PACKAGE	CLASSES	JOB
controller/	ReservationController, ShowController, TokenController	Turn HTTP into calls, and results into status codes

service/	ReservationService, ShowService, HoldSweeper, ReserveCommand, ReservationTransition	Transactions, decisions and their log lines; the service's input type and lifecycle rules
event/	ReservationCreated, ReservationReplayed, ReservationDeclined, HoldConfirmed, ReservationReleased	What happened, as plain records
exception/	ApiException, ReservationDeclinedException, DeclineReason	How a request is refused: reason and HTTP status
repository/	ReservationRepository, SeatRepository, UserSeatCountRepository, ShowRepository	All SQL, one guarantee per method
metrics/	ReservationMetrics, SeatGauges, HoldExpiryGauge	Prometheus counters, per-show seat gauges, and the overdue-holds alarm